

PROMPT ENGINEERING — LABORATORY MANUAL

Elaborated Application-Based Exercises 6 – 10

Prompting Strategies · Applied Prompt Design · LLM API Programming · Code & SQL Generation · Comparative Evaluation

Field	Details
Course	Prompt Engineering / Generative AI Laboratory
Exercises Covered	6, 7, 8, 9, 10
Mode	Hands-on laboratory with real-time application scenarios
Tools Required	Python 3.10+, an LLM API (OpenAI / Google Gemini / Hugging Face), SQLite, pandas, openpyxl
Student Name	S.DHARANI
Register Number	192472364
Date	12-08-2026

Index of Exercises

Ex. No.	Title	Real-Time Application Domain
6	Design and compare Zero-shot, One-shot and Few-shot prompts	Customer-support ticket triage — e-commerce
7	Prompts for summarization, professional email and content creation	Corporate communications — manufacturing firm
8	Text generation using an LLM API	Agri-advisory SMS helpline / placement notifications
9	Generating Python programs and SQL queries using structured prompts	Retail chain sales analytics
10	Compare and evaluate responses across prompting strategies	Banking FAQ assistant — pre-production evaluation

Exercise 6 — Design and Compare Zero-shot, One-shot and Few-shot Prompts

Aim

To design three functionally equivalent prompts using Zero-shot, One-shot and Few-shot strategies for a single real-world classification task, and to compare them quantitatively on accuracy, output-format reliability, token cost and latency.

Real-Time Application Scenario

Automated customer-support ticket triage for an e-commerce company. An online retailer receives approximately 5,000 support messages every day. Each incoming message must be automatically routed to one of five queues — Delivery Delay, Payment/Refund, Product Defect, Account Access, or Feedback/Other — and simultaneously tagged with an urgency level (High / Medium / Low) and a sentiment label. Human triage costs the company roughly 40 agent-hours a day, so the prompting strategy chosen here directly determines both operating cost and customer response time.

Prerequisites

- Working access to any one LLM (web interface is sufficient for this exercise; API preferred).
- A spreadsheet for recording observations.
- Basic understanding of in-context learning and token accounting.

Procedure

1. **Build the test set.** Collect or compose 15 raw customer messages. Deliberately include difficult cases: multi-category messages, sarcasm, very short messages (“where???”), code-mixed Tanglish/Hinglish text, and messages containing an order ID that must not be echoed back.
2. **Create the gold labels.** Manually assign the correct category, urgency and sentiment to all 15 messages before running any prompt. This is your ground truth; do not revise it after seeing model output.
3. **Design Prompt A (Zero-shot).** Provide only the instruction, the five category definitions and the required output schema. No examples.
4. **Design Prompt B (One-shot).** Same instruction, plus exactly one fully worked example demonstrating the exact JSON output format.
5. **Design Prompt C (Few-shot).** Same instruction, plus four to six examples chosen to cover the edge cases identified in Step 1. Balance the examples across categories.
6. **Control the variables.** Fix temperature at 0.2, top-p at 1.0 and max tokens at 150 for every run, so that prompt design is the only variable that changes.
7. **Execute.** Run all 15 messages through each of the three prompts (45 runs total). Record the raw output verbatim, including any malformed responses.
8. **Measure.** Compute category accuracy, urgency accuracy, JSON validity rate, mean prompt tokens, mean completion tokens and mean latency for each strategy.

Sample Prompt Skeleton (Few-shot)

```
You are a support-ticket triage engine for an e-commerce company.
Classify the customer message into exactly one CATEGORY from:
  DELIVERY_DELAY | PAYMENT_REFUND | PRODUCT_DEFECT | ACCOUNT_ACCESS | FEEDBACK_OTHER
Also assign URGENCY (HIGH|MEDIUM|LOW) and SENTIMENT (POSITIVE|NEUTRAL|NEGATIVE).
Return ONLY valid JSON. No explanation, no markdown fences.

Message: "Ordered on the 3rd, still not shipped, I need it for a wedding."
Output: {"category":"DELIVERY_DELAY","urgency":"HIGH","sentiment":"NEGATIVE"}

Message: "Refund shows credited but nothing in my bank account since 9 days."
Output: {"category":"PAYMENT_REFUND","urgency":"HIGH","sentiment":"NEGATIVE"}

Message: "<<TEST MESSAGE HERE>>"
Output:
```

Observation Table

Metric	Zero-shot	One-shot	Few-shot	Best	Remarks
Category accuracy (/15)	14/15	14/15	14/15	ALL equal	All three strategies achieved the same category accuracy of 14/15..
Urgency accuracy (/15)	9/15	7/15	8/15	Zero-shot	Zero-shot achieved the highest urgency accuracy, performing better than One-shot and Few-shot.
Valid JSON rate (%)	100%	100%	100%	ALL EQUAL	All three strategies produced valid JSON for every test case.
Mean prompt tokens	271.67	321.67	521.67	ZERO-SHOT	Zero-shot uses the smallest prompt because it contains no in-context examples.
Mean completion tokens	24.47	20.00	19.53	FEW-SHOT	Few-shot produced the lowest average completion-token usage in this experiment. [
Mean latency (ms)	4956.99	4354.60	4578.91	ONE-SHOT	One-shot had the lowest measured latency among the three strategies.
Cost per 5,000 calls	0	0	0	ALL EQUAL	All strategies have 0 API cost because the experiment was performed locally using Ollama.

Extension Experiment — Majority-Label Bias

Repeat the Few-shot run with all six in-context examples drawn from a single category (say `PAYMENT_REFUND`). Re-score the 15 test messages and observe how strongly the model over-predicts that category. Report the shift in the confusion matrix. This demonstrates that in-context examples act as a prior, not merely as a format guide, and explains why example selection must be class-balanced in production.

Expected Result and Discussion Points

- Zero-shot is cheapest per call but usually the least reliable on output format.
- One-shot typically fixes formatting almost entirely at very little token cost — often the highest return per token spent.
- Few-shot gives the best accuracy on edge cases but multiplies prompt tokens on every single call, which at 5,000 calls/day is a real recurring cost.
- Justify which strategy you would actually deploy, taking accuracy, cost and latency together rather than accuracy alone.

Exercise 7 — Prompts for Text Summarization, Professional Email Generation and Content Creation

Aim

To develop structured, constraint-driven prompts for three distinct business writing tasks and to determine, through ablation, which prompt components most influence output quality.

Real-Time Application Scenario

Communications workflow of a mid-size manufacturing firm. The firm must convert long production-review meetings into circulated minutes, notify clients of supply disruptions without creating legal exposure, and launch new products across multiple content channels — all with consistent tone and factual accuracy.

Part 7A — Text Summarization: Minutes-of-Meeting Condenser

Input: a 900–1,200 word transcript of a production review meeting (any long article or transcript may be substituted).

From the same source text, design prompts that produce three different summaries:

1. **Executive abstract** — maximum 80 words, no technical jargon, written for the Managing Director.
2. **Action-item list** — bulleted, each item carrying Owner, Task and Deadline.
3. **Technical summary** — retains machine names, defect counts and all numerical values.

Measurements: compression ratio (output tokens ÷ input tokens) and hallucination count — any number, name or date appearing in the summary that is not present in the source. Verify hallucinations manually; this is the single most important quality check in summarization.

Part 7B — Professional Email Generation: Client Delay Notification

Scenario: a shipment of 400 units is delayed by nine days because of a supplier failure. The client is a long-standing account and is likely to be annoyed.

Build the prompt using an explicit Role–Context–Task–Tone–Constraint structure:

```
ROLE:      You are a senior account manager at a precision-components manufacturer.
CONTEXT:    Client = Meridian Auto (7-year account). PO-4471, 400 units.
            Original date 12 Sep. New date 21 Sep. Cause: sub-supplier casting failure.
TASK:       Write the delay notification email.
TONE:       Professional, accountable, not grovelling.
CONSTRAINTS:
- Maximum 150 words.
- Must state the revised delivery date explicitly.
- Must offer exactly one concrete remedy (partial early dispatch OR expedited freight).
- Must NOT admit legal liability or reference penalty clauses.
- Must NOT apologise more than twice.
OUTPUT:     Subject line, then body. No placeholders like [Name].
```

Generate three tone variants — formal, empathetic and assertive. Have three peers rank which version they would actually send, and record the disagreement.

Part 7C — Content Creation: Product Launch Campaign

From one structured prompt, generate a coordinated content set for a new energy-efficient industrial motor:

- A LinkedIn post in B2B tone (specification-led, 120–150 words).
- An Instagram caption in consumer tone with hashtags (≤ 40 words).
- A 60-word website blurb incorporating three SEO keywords that you supply.

All three must remain factually consistent with the same product datasheet. Check for contradictions across channels — a very common failure in multi-output generation.

Ablation Study (Mandatory Analysis)

Re-run the email prompt five times, removing exactly one component each time, and score the degradation:

Component Removed	Observed Effect on Output	Severity (1–5)
Role	Less role-specific and less authoritative response; tone may become generic.	2
Context (facts/dates)	Important facts, dates or order details may become incorrect or missing	5
Tone specification	Email tone becomes less controlled and may sound too casual or too apologetic.	3
Word-count constraint	Output may become unnecessarily long and less concise.	2
Output-format spec	Subject/body structure may become inconsistent or include extra explanations.	3

Evaluation Table

Task	Length Adherence	Tone Match (1–5)	Factual Fidelity (1–5)	Hallucinations
Executive abstract	≤ 80 words	5	5	0
Action-item list	Required format	5	5	0
Technical summary	Within limit	5	5	1
Email — formal	≤ 150 words	4	5	1
Email — empathetic	≤ 150 words	5	5	0
Email — assertive	≤ 150 words	5	4	0
LinkedIn post	120-150 words	3	5	1
Instagram caption	≤ 40 words	5	5	0
Website blurb	60 words	3	5	1

Exercise 8 — Text Generation Using an LLM API

Aim

To build a production-shaped Python application that calls an LLM API (OpenAI, Google Gemini or Hugging Face Inference API) for controlled text generation, and to study the effect of decoding parameters, cost and failure handling.

Real-Time Application Scenario

Personalised advisory-message generator for a farmer support helpline. The system accepts structured input — crop, district, soil type, current weather and the farmer's preferred language — and generates a short advisory SMS of under 160 characters in that language. An alternative scenario, if preferred, is a college placement-notification generator that personalises drive announcements per student branch and CGPA band.

Procedure

1. **Configure API access securely.** Store the key in an environment variable and read it with `os.environ`. Never hard-code a key in source, and never commit it to version control. State this explicitly in your report as a security practice.
2. **Implement the generator.** Write `generate_advisory(params: dict) -> str`, which fills a prompt template from the structured input and calls the API.
3. **Run a parameter sweep.** Generate the same advisory at temperature = 0.0, 0.4 and 0.9, and at top_p = 0.5 and 1.0. Record how the output changes and whether factual content drifts at high temperature.
4. **Instrument the code.** Log latency in milliseconds, prompt tokens, completion tokens and estimated cost for every call.
5. **Add production-grade error handling.** Wrap the call in `try/except`; retry with exponential backoff on HTTP 429 (rate limit); fall back to a static template message if the API is unavailable after three attempts.
6. **Batch generate.** Produce advisories for ten different farmer profiles and write the results to a CSV with all logged metrics.

Reference Code Skeleton

```
import os, time, csv, json

PROMPT = (
    "You are an agricultural extension officer.\n"
    "Write ONE advisory SMS under 160 characters in {language}.\n"
    "Crop: {crop} | District: {district} | Soil: {soil} | Weather: {weather}\n"
    "Rules: one actionable instruction, no greetings, no emojis,"
    " no chemical dosage without units, plain words only."
)

def generate_advisory(params: dict, temperature: float = 0.3) -> dict:
    prompt = PROMPT.format(**params)
    for attempt in range(3):
```



```

try:
    t0 = time.time()
    # ---- replace with your provider's call ----
    resp = client.messages.create(model=MODEL, max_tokens=120,
                                  temperature=temperature,
                                  messages=[{'role': 'user', 'content': prompt}])
    latency = (time.time() - t0) * 1000
    return {'text': resp.content[0].text.strip(),
            'latency_ms': round(latency, 1),
            'in_tokens': resp.usage.input_tokens,
            'out_tokens': resp.usage.output_tokens}
except RateLimitError:
    time.sleep(2 ** attempt)
except Exception as e:
    print('error:', e); break
return {'text': FALLBACK_TEMPLATE.format(**params), 'latency_ms': 0,
        'in_tokens': 0, 'out_tokens': 0}

```

Observation Table — Parameter Sweep

Temp	top_p	Output (first 40 chars)	Chars	Latency (ms)	Factually safe?
0.0	1.0	Thanjavur farmers: Heavy rain expected.	121	750.05	YES
0.4	1.0	Thanjavur farmers: Heavy rain expected.	126	908.17	YES
0.9	1.0	Thanjavur farmers: Heavy rain expected.	126	675.15	YES
0.4	0.5	Thanjavur farmers: Heavy rain expected.	118	773.56	YES
0.9	0.5	Thanjavur farmers: Heavy rain expected.	113	1043.39	YES

Discussion

Conclude with a written paragraph on when a deterministic template beats an LLM. For an SMS advisory carrying pesticide instructions, variability is a defect rather than a feature — this is a genuinely important engineering conclusion, and the exercise is not complete without it.

Exercise 9 — Generating Python Programs and SQL Queries Using Structured Prompts

Aim

To generate executable SQL queries and Python modules from structured, schema-grounded prompts, to verify every generated artefact by execution, and to measure first-attempt correctness.

Real-Time Application Scenario

Analytics layer for a retail chain's sales database. Business users ask questions in plain English; the system must translate them into correct SQL against a fixed schema and produce a scheduled Excel report. Any hallucinated column name causes a production failure, which makes schema grounding the central lesson of this exercise.

Database Schema (supply this inside every prompt)

```
customers(cust_id, name, city, join_date, segment)
products(prod_id, name, category, unit_price)
orders(order_id, cust_id, order_date, status)
order_items(order_id, prod_id, qty, discount_pct)
```

Part 9A — SQL Generation

Write structured prompts that generate queries for these genuine business questions:

1. Month-over-month revenue growth per category for the last twelve months.
2. Top five customers by lifetime value, excluding cancelled orders.
3. Products whose sales dropped by more than 30% quarter-over-quarter.
4. Customers who purchased in Q1 but not in Q2 (churn candidates).

Each prompt must contain: the schema block, the SQL dialect (SQLite or MySQL), the business-rule definitions (for example, $\text{revenue} = \text{qty} \times \text{unit_price} \times (1 - \text{discount_pct}/100)$; valid orders exclude `status = 'CANCELLED'`), the output format instruction (“return only SQL, no explanation”), and the coding constraints (no `SELECT *`, use CTEs, alias every aggregate).

Verification step (mandatory). Create the four tables in SQLite, insert about 30 sample rows, and actually execute every generated query. A query that was never run must be reported as unverified, not as correct. Log each failure and its cause.

Common failure modes to watch for and report:

- Invented column names not present in the supplied schema.
- Date functions borrowed from the wrong dialect (`DATE_SUB` in SQLite, `strtime` in MySQL).
- Silently ignoring the status filter, inflating revenue.
- Discount applied additively instead of multiplicatively.

Part 9B — Python Program Generation

Prompt the model to produce a module `sales_report.py` that connects to the SQLite database, computes a category-wise revenue summary and exports it to Excel with a bar chart.

Constraints to place in the prompt: type hints on all functions, docstrings, PEP-8 compliance, no external database libraries beyond sqlite3 and pandas, parameterised queries only (no string concatenation into SQL), and an `if __name__ == "__main__":` guard.

Iterative refinement. Run the generated code. Paste the resulting traceback back into a second-pass prompt and ask for a corrected version. Repeat until it executes. Preserve prompt versions v1 → v2 → v3 in your record — the refinement trail is part of the deliverable.

Observation Table

Artefact	Runs on 1st try?	Attempts to fix	Error class	Root cause
Query 1 — MoM growth	Yes	0	no	Correct schema, revenue formula and date logic
Query 2 — Top 5 LTV	Yes	0	no	Correct joins and cancellation filter
Query 3 — QoQ drop	Yes	0	no	Correct quarterly revenue comparison
Query 4 — Churn	no	1	sqlite3.ProgrammingError	Database was closed before query execution
sales_report.py	Yes	0	no	Program executed successfully

Result and Discussion

Compute first-attempt correctness rate separately for SQL and Python and compare the two. Explain, with evidence from your own error log, why placing the schema inside the prompt drastically reduces hallucinated columns, and why executing generated code is the only acceptable form of verification.

Exercise 10 — Comparative Evaluation of Prompting Strategies

Aim

To evaluate five prompting strategies on one fixed task using a rubric, human inter-rater agreement and cost/latency measurement, and to produce a deployment recommendation.

Real-Time Application Scenario

Selecting a prompting strategy for a bank's FAQ assistant before production rollout. The assistant must answer customer questions on loan eligibility, KYC requirements and service charges strictly from an approved policy document. A hallucinated interest rate or fee is a regulatory issue, not merely a quality issue, so the evaluation must measure hallucination explicitly.

Fixed Task

Answer the same twelve customer questions using each of five strategies, holding the model, temperature and question order constant.

S. No.	Strategy	Prompt Characteristic
1	Zero-shot	Instruction only, no examples, no reasoning cue
2	Few-shot	Four worked Q&A pairs in the approved house style
3	Chain-of-Thought	“Reason step by step, then give the final answer”
4	Role / Persona	“You are a compliance-trained bank officer...”
5	Grounded (retrieval-style)	Policy text pasted in; answer only from the passage, else say “not covered in policy”

Evaluation Rubric (score each response 1–5)

Dimension	What You Check
Factual accuracy	Answer matches the policy document exactly
Completeness	Every part of a multi-part question is addressed
Hallucination	Count of invented facts, fees, rates or clauses (lower is better)
Tone and compliance	No financial promises, no advice beyond stated policy
Conciseness	Usable as a chat reply, under 120 words

Procedure

1. Generate all 60 responses (12 questions × 5 strategies) and store them anonymised, so scorers cannot tell which strategy produced which answer.
2. Have two classmates score independently against the rubric.

3. Compute simple inter-rater agreement (percentage agreement, or Cohen's kappa if you have covered it). This teaches that LLM evaluation is a measurement problem, not a matter of opinion.
4. Optionally add an automatic metric such as ROUGE-L against reference answers, and comment specifically on the cases where the automatic score disagrees with human judgement.
5. Record mean cost and mean latency per strategy alongside the quality scores.

Consolidated Results Matrix

Strategy	Accuracy	Complete	Halluc.	Tone	Concise	Tokens / Latency
Zero-shot	5.0	4.9	5.0	5.0	4.9	827.90
Few-shot	5.0	4.8	5.0	4.9	4.9	1007.56
Chain-of-Thought	5.0	5.0	5.0	5.0	4.7	1097.65
Role / Persona	5.0	4.9	5.0	5.0	4.9	939.41
Grounded	5.0	5.0	5.0	5.0	5.0	1212.34

Deliverables

- The completed 5 × 5 matrix plus cost and latency columns.
- A bar chart of mean scores per strategy per dimension.
- Inter-rater agreement figure, with a note on which dimension scorers disagreed about most (usually tone).
- A one-page recommendation stating **which strategy ships**, the fallback behaviour when the model is uncertain, and the guardrail you would add for the residual hallucination rate.

Expected Conclusion

Chain-of-Thought typically wins on accuracy but loses badly on tokens and response time; grounded prompting usually wins on hallucination, which is the dimension that actually governs regulatory risk. The exercise should end by demonstrating that the highest-scoring strategy on quality alone is not automatically the one a product team should deploy.